

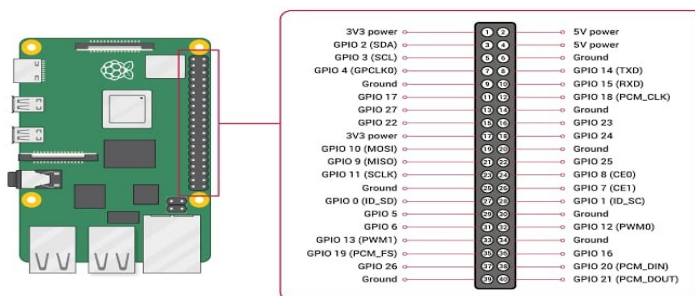
Nile University (NU) IoT Research Internship Report

Day 1

Overview

Day 1 was split between two tracks: a software warm-up exercise in C (the Emirp Number Finder) and the start of hands-on IoT work with the Raspberry Pi, beginning with system setup and basic GPIO control.

Task 1: Raspberry Pi Setup



Installed the Raspberry Pi Imager and used it to flash the Linux operating system onto the SD card. Inserted the SD card into the Raspberry Pi, which was then ready to boot and use. This was the foundation step for all later IoT tasks, since none of the GPIO or hardware work could begin without a working operating system on the board.

Then we executed a simple Hello World example in the terminal.

The first screenshot shows the terminal output of the Raspberry Pi Imager's command-line interface. It displays the progress of the OS installation, including the selection of the target board (Raspberry Pi 4) and the target OS (Raspberry Pi OS). The second screenshot shows the terminal output of the 'Hello World' program. The user has created a file named 'hello.c' and executed it, resulting in the output 'Hello World'.

Task 2: Emirp Number Finder (C Program)

Wrote a C program to find emirp numbers (primes whose digit-reversal is also prime, excluding palindromes) within a user-input range [A, B]. Key elements:

- Primality testing optimized to $\sqrt{n}$ using `math.h`
- Dynamic memory allocation (`malloc`) to store emirp pairs and their differences

- Output of each emirp with its reversed counterpart and absolute difference
- Summary statistics: total count, largest difference, smallest difference

This task was independent of the Pi setup but served as a refresher on C fundamentals (memory management, modular functions, optimization) that would carry over into the GPIO programming later in the day, since that work is also done in C.

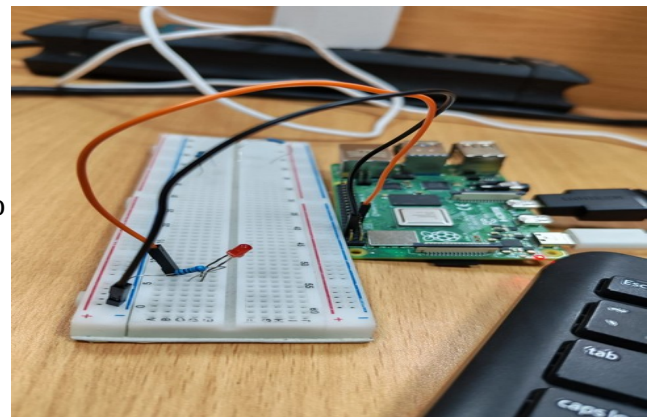
```
Intern@Intern:~/tasks $ ./task1
Enter A and B
1
501

Emirp numbers from 1 to 501:
13, 31 (diff = 18)
17, 71 (diff = 54)
31, 13 (diff = 18)
37, 73 (diff = 36)
71, 17 (diff = 54)
73, 37 (diff = 36)
79, 97 (diff = 18)
97, 79 (diff = 18)
113, 311 (diff = 198)
311, 113 (diff = 198)

Summary:
Total emirps found: 10
Largest difference: 198 (achieved by 113,311)
Smallest difference: 18 (achieved by 13,31,79,97)
Intern@Intern:~/tasks $
```

Task 3: LED Control via Terminal

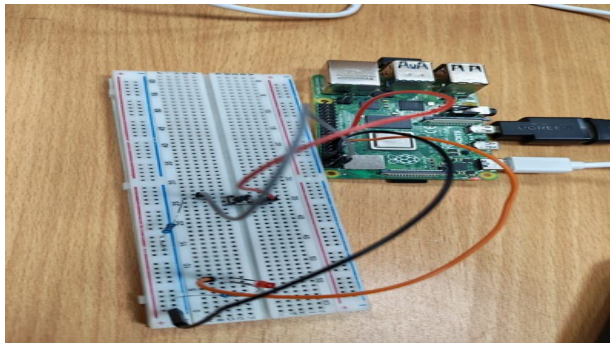
Lit an LED connected to the Raspberry Pi by controlling a GPIO pin directly from the terminal. This was the first practical test that the OS setup from Task 1 was working correctly and that the board could interact with external hardware. It also introduced the `libgpiod` library for GPIO control, which required compiling with the `-lgpiod` flag (initial compilation failed until this flag was added, which was a useful debugging lesson).



Task 4: LED Control via Button (Polling)

Extended the static LED control from Task 3 into an interactive circuit: the LED could now be turned on and off using a physical button. This was implemented using a polling approach, where the program continuously checks the button's state in a loop.

Improvement over Task 3: Task 3 only proved the Pi could drive an output (the LED). Task 4 added an input (the button) and combined it with the output, which is the basic building block of interactive IoT devices.



Task 5: LED Control via Button (Timer Service)

Reimplemented the button-LED interaction from Task 4, this time using a timer service instead of continuous polling.

Improvement over Task 4: polling is inefficient because it keeps the CPU busy checking the button state constantly. Using a timer service instead is a step toward more efficient, event-driven hardware control, which is closer to how real IoT systems are designed to conserve processing resources.

```

36 int main(void) {
37     struct gpiochip *chip;
38     struct sigaction sa;
39     struct itimerval timer;
40
41     // 2. Open the main Raspberry Pi 4 controller chip
42     chip = gpiochip_open_by_name(CHIP_NAME);
43     if (!chip) {
44         perror("Open chip failed");
45         return 1;
46     }
47
48     // 3. Configure the Output LED Line
49     led_line = gpiochip_get_line(chip, LED_PIN);
50     if (!led_line || gpiochip_line_request_output(led_line, "timer_led", 0) < 0) {
51         perror("LED line configuration failed");
52         gpiochip_close(chip);
53         return 1;
54     }
55
56     // 4. Configure the Input Button Line (Sets it to listen for high signals)
57     button_line = gpiochip_get_line(chip, BUTTON_PIN);
58     if (!button_line || gpiochip_line_request_input(button_line, "button_sensor") < 0) {
59         perror("Button line configuration failed");
60         gpiochip_close(chip);
61         return 1;
62     }
63
64     // 5. Register the Background Interrupt Signal Handler
65     sa.sa_handler = &timer_handler;
66     sa.sa_flags = SA_RESTART;
67     sigaction(SIGALRM, &sa, NULL);
68
69     // 6. Set Blinking Speed (200,000 microseconds = 200ms for a faster blink)
70     timer.it_interval.tv_sec = 0;
71     timer.it_interval.tv_usec = 200000; // Continuous repeat interval
72     timer.it_value.tv_sec = 0;
73     timer.it_value.tv_usec = 200000; // First initial delay time
74
75     // 7. Arm the interval timer
76     setitimer(ITIMER_REAL, &timer, NULL);
77
78     printf("System active! Hold down the button to blink the LED.\n");
79     printf("The main thread is 100% free. Press Ctrl+C to exit...\n");
80
81     // 8. The Main Loop is completely empty and non-blocking!
82     while (1) {
83         pause(); // Puts the main thread to sleep to use 0% CPU until a timer tick occurs
84     }
85
86     // Clean up hardware handles on exit
87     gpiochip_line_release(button_line);
88     gpiochip_line_release(led_line);
89     gpiochip_close(chip);
90     return 0;
91 }

```

Key Concepts Used

- Prime number algorithms with optimization (sqrt(n) primality test)
- Dynamic arrays and structs (EmirpData)
- Memory management (malloc/free)
- Raspberry Pi OS installation and SD card flashing
- GPIO control using the libgpiod library
- Digital input handling (button) and output handling (LED)

- Polling vs. timer-based (event-driven) control
- Compilation with correct library flags (-lm, -lgpiod)

Outcome

- Successfully set up the Raspberry Pi and got it ready for development
- Implemented the emirp finder with correct input validation and summary output
- Learned basic GPIO pin control for LED blinking, including resolving a compilation issue caused by a missing library flag
- Progressed from simple output control (LED) to input-output interaction (button + LED) to a more efficient control method (timer service instead of polling)
- Set up a `gccall` bash function to automate compilation with the correct library flags for future tasks